

GPUMODE Lecture Study Notes: Lecture 88: TinyTPU

Core Problem the Lecture Addresses

This lecture studies how a tiny tensor-processing chip can be designed, fabricated, verified, and connected all the way up to PyTorch. The central problem is:

How can one build a real, micron-scale, TPU-like accelerator under extreme area and I/O constraints, while still preserving the essential ideas of systolic matrix multiplication, low-precision arithmetic, streaming execution, and machine-learning integration?

The speaker designed a small application-specific integrated circuit, or ASIC, through the Tiny Tapeout flow. The chip implements a minimal tensor-processing unit around a 2×2 systolic array. Although the chip is tiny compared with industrial TPUs, it demonstrates the same core stack:

- hardware architecture with processing elements, registers, control logic, and I/O pins;
- RTL design in Verilog;
- physical design through open-source CAD tooling;
- timing and manufacturability verification;
- integer and FP8/BF16-inspired low-precision arithmetic;
- tiled matrix multiplication kernels;
- quantization-aware training;
- PyTorch graph rewriting and custom backend integration.

The chip is constrained to an area of approximately $320 \times 100 \mu\text{m}$, so it cannot implement a large industrial systolic array such as 128×128 . Instead, it uses a 2×2 systolic array as the primitive compute tile. Larger matrix multiplications are decomposed into many 2×2 block multiplications.

Required Background

ASICs and Tiny Tapeout

An ASIC is an **application-specific integrated circuit**: a chip designed for a particular computation rather than general-purpose programmability. A TPU is an ASIC specialized for tensor computations such as matrix multiplication.

Tiny Tapeout provides an accessible chip fabrication path. Instead of one designer paying for an entire wafer or die, many small designs share a multi-project wafer. Each participant receives a small rectangular tile of silicon area.

In this lecture, one Tiny Tapeout tile is approximately

$$100\ \mu\text{m} \times 160\ \mu\text{m}.$$

The TinyTPU design used two such tiles, giving approximately

$$320\ \mu\text{m} \times 100\ \mu\text{m}.$$

The process node used is SkyWater 130 nm, often called Sky130. This is far less dense than modern advanced nodes such as 5 nm, 3 nm, or 2 nm, but it is accessible through open-source tooling.

From Verilog to Silicon

The design begins as RTL, or register-transfer-level code, written in Verilog. RTL describes digital circuits using:

- **combinational logic**, whose outputs depend immediately on current inputs;
- **sequential logic**, whose state updates on clock edges through flip-flops;
- explicit registers, wires, arithmetic operators, finite-state machines, and control paths.

The simplified chip-design pipeline is:

1. write Verilog RTL;
2. run simulation and functional tests;
3. synthesize RTL into gates;
4. place standard cells on the chip area;
5. route wires between placed cells;
6. verify timing, area, and manufacturability;
7. produce a GDSII layout file;

8. send the GDSII file to the factory for fabrication.

The lecture mentions OpenROAD as the open-source CAD flow that performs much of the physical design.

Matrix Multiplication

The chip accelerates matrix multiplication. For matrices W , X , and C ,

$$C = WX.$$

For a 2×2 multiplication,

$$W = \begin{bmatrix} w_{00} & w_{01} \\ w_{10} & w_{11} \end{bmatrix}, \quad X = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix},$$

the output is

$$C = \begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix},$$

where

$$c_{00} = w_{00}x_{00} + w_{01}x_{10}, \tag{1}$$

$$c_{01} = w_{00}x_{01} + w_{01}x_{11}, \tag{2}$$

$$c_{10} = w_{10}x_{00} + w_{11}x_{10}, \tag{3}$$

$$c_{11} = w_{10}x_{01} + w_{11}x_{11}. \tag{4}$$

The TinyTPU implements this operation in hardware.

Systolic Arrays

A systolic array is a regular grid of processing elements, or PEs. Each PE performs a small operation, usually multiply-accumulate, while data flows rhythmically through the array.

A typical PE performs

$$\text{acc} \leftarrow \text{acc} + ab.$$

The term **systolic** comes from the idea of pulsed movement, like a heartbeat. Data moves one step per clock cycle through a structured array.

The key benefit is data reuse. Instead of loading every operand repeatedly from global memory or external I/O, each value is reused as it moves through neighboring PEs.

Main Concepts

Overall TinyTPU Architecture

The chip contains:

- a 2×2 systolic array;
- small register or SRAM-like buffers for matrix elements;
- a control unit;
- fixed-width input and output ports;
- instruction-like signals for fused operations;
- clock and reset lines.

The Tiny Tapeout platform provides a fixed I/O interface:

- 8 dedicated input pins;
- 8 dedicated output pins;
- 8 bidirectional pins;
- a clock;
- a reset.

Because the dedicated input interface is only 8 bits wide, only one 8-bit matrix element can be loaded per cycle through that path. This creates a major I/O bottleneck.

Why Only a 2×2 Systolic Array?

Industrial TPUs may use large systolic arrays, such as 128×128 or larger. TinyTPU cannot do this because the available silicon area is extremely small.

If the array dimension is $N \times N$, then the number of PEs scales as

$$N^2.$$

Each PE needs:

- multiplier logic;
- adder logic;

- local accumulator state;
- input and output registers;
- routing wires to neighboring PEs;
- control signals.

Thus, increasing from 2×2 to 128×128 changes the PE count from

$$2^2 = 4$$

to

$$128^2 = 16384.$$

The PE count multiplier is

$$\frac{16384}{4} = 4096.$$

That ignores additional routing, buffering, timing, power, clock-tree, and verification complexity. Therefore, the lecture focuses on a minimal array that still expresses the architectural principle.

Processing Element Design

Each PE accepts two 8-bit inputs, multiplies them, and accumulates locally. The lecture emphasizes that the TinyTPU accumulates inside each PE rather than forwarding partial sums to the next PE.

A simplified PE computation is:

$$\text{acc}_{ij}^{(t+1)} = \text{acc}_{ij}^{(t)} + w_{ik}^{(t)} x_{kj}^{(t)}.$$

The PE also forwards its operands:

$$w \rightarrow \text{right neighbor}, \quad x \rightarrow \text{bottom neighbor}.$$

This means weights flow horizontally and activations flow vertically.

Skewed Input Schedule

For matrix multiplication, operands are not all injected at the same time. They are skewed so that the correct pair of operands meets at the correct PE in the correct cycle.

For 2×2 multiplication, the rough pattern is:

- cycle 0: top-left PE receives w_{00} and x_{00} ;
- later cycles: weights move right and inputs move down;
- each PE accumulates the products corresponding to one output element;
- after four systolic cycles, all four output accumulators hold the output tile.

Each PE corresponds to one output coordinate:

$$\text{PE}_{ij} \longleftrightarrow c_{ij}.$$

Thus, for the 2×2 array:

$$\begin{bmatrix} \text{PE}_{00} & \text{PE}_{01} \\ \text{PE}_{10} & \text{PE}_{11} \end{bmatrix} \longleftrightarrow \begin{bmatrix} c_{00} & c_{01} \\ c_{10} & c_{11} \end{bmatrix}.$$

Visual Concept

Draw a 2×2 PE grid. Show W values entering from the left and moving right, while X values enter from the top and move downward. Inside each PE, draw a small multiply-add accumulator.

Instruction-Like Control Signals

The chip includes control signals that behave like very small custom instructions. The lecture mentions signals such as:

- enable;
- load weights;
- weight stationary mode;
- fused transpose;
- fused ReLU;
- elementwise operation mode.

These are not full general-purpose CPU instructions. They are direct hardware control lines that alter the datapath behavior.

For example, fused transpose changes the loading pattern of the second matrix. If the original operation is

$$C = WX,$$

then fused transpose allows

$$C = WX^T.$$

In a 2×2 matrix,

$$X = \begin{bmatrix} x_{00} & x_{01} \\ x_{10} & x_{11} \end{bmatrix}, \quad X^T = \begin{bmatrix} x_{00} & x_{10} \\ x_{01} & x_{11} \end{bmatrix}.$$

The hardware can implement this by swapping the loading order of x_{01} and x_{10} , rather than by performing a separate transpose pass.

Fused ReLU

The chip can also fuse ReLU. ReLU is

$$\text{ReLU}(z) = \max(z, 0).$$

For an output matrix C , elementwise ReLU is

$$Y_{ij} = \max(C_{ij}, 0).$$

In hardware, this is cheap compared with matrix multiplication. It is essentially a sign check or comparator followed by a mux:

$$Y_{ij} = \begin{cases} C_{ij}, & C_{ij} > 0, \\ 0, & C_{ij} \leq 0. \end{cases}$$

The important systems idea is that fusion avoids an extra round trip through memory or I/O. Even if ReLU is simple, doing it separately may require reading and writing an entire matrix again.

Hardware-Level Explanation

I/O Bottleneck

The chip has only 8 dedicated input bits per cycle and 8 dedicated output bits per cycle. This is a hard bottleneck because the systolic array would ideally like to receive multiple operands per cycle.

For a 2×2 systolic array, a full-rate feed would require:

$$2 \text{ weights} + 2 \text{ activations} = 4 \text{ values per cycle.}$$

If each value is 8 bits, then the ideal feed bandwidth is

$$4 \times 8 = 32 \text{ bits per cycle.}$$

But the dedicated input interface provides only

$$8 \text{ bits per cycle.}$$

Therefore, the array cannot be fed naively at full bandwidth every cycle. The chip must overlap loading, computation, and output streaming.

Streaming Execution Pattern

The baseline streaming pattern is:

1. load eight 8-bit values for two 2×2 matrices;
2. begin systolic computation before the entire loading phase finishes;
3. compute the 2×2 output tile;
4. stream out four 16-bit output values over the 8-bit output port;
5. while outputting one tile, load the next input tile.

Two 2×2 input matrices contain

$$4 + 4 = 8$$

elements. With one 8-bit element loaded per cycle, loading takes

$$8 \text{ cycles.}$$

The output matrix contains four 16-bit values:

$$4 \times 16 = 64 \text{ bits.}$$

With one 8-bit output port, output takes

$$\frac{64}{8} = 8 \text{ cycles.}$$

This symmetry is useful: while one result tile is being output over 8 cycles, the next pair of input tiles can be loaded over 8 cycles.

Why Computation Starts Before Loading Finishes

The systolic array does not need all matrix elements at cycle 0. The first multiply only needs w_{00} and x_{00} . Therefore, once enough elements have arrived to sustain the skewed schedule, computation can begin.

This is a small-scale example of **pipeline overlap**. Instead of doing:

load all $\rightarrow$ compute $\rightarrow$ store all,

the chip tries to do:

load tile $n + 1$ $\parallel$ compute tile n $\parallel$ store tile $n - 1$.

The notation $\parallel$ means these phases overlap in time.

Weight Stationary Mode

The lecture later introduces an improved scheme where the first matrix, treated as weights, can remain resident in local memory. This is a form of **weight stationary** dataflow.

Suppose a larger matrix multiplication is blocked into 2×2 tiles:

$$C_{ij} = \sum_k A_{ik} B_{kj}.$$

For a fixed A_{ik} , the same tile may be multiplied with several B_{kj} tiles. Reloading A_{ik} every time wastes I/O bandwidth.

A weight-stationary dataflow keeps A_{ik} on chip while streaming different B_{kj} tiles. This reduces input traffic.

If the weight tile is reused R times, then the amortized weight-loading cost per use is

$$\frac{4 \text{ elements}}{R}.$$

As R grows, the cost approaches zero.

Using Bidirectional Pins

The chip also has 8 bidirectional pins. The lecture notes that these were initially not used heavily because coordinating them complicates the control logic. Later, the speaker experiments with using them for output as well.

If both dedicated output pins and bidirectional pins are used as output channels, the output bandwidth becomes

$$8 + 8 = 16 \text{ bits per cycle.}$$

Then a 64-bit output tile can be emitted in

$$\frac{64}{16} = 4 \text{ cycles.}$$

This matches the four input elements needed when weights are stationary and only the second matrix tile must be loaded. This is why the improved scheme can roughly double throughput.

Clocking, Flip-Flops, and Timing

The chip is synchronous. State is stored in flip-flops. A flip-flop samples its input on a clock edge and exposes the sampled value on its output.

For correct timing, data launched from one flip-flop must arrive at the next flip-flop before the next sampling edge. The basic setup-time condition is:

$$T_{\text{clk}} \geq T_{\text{clk-to-q}} + T_{\text{logic}} + T_{\text{route}} + T_{\text{setup}}.$$

Here:

- T_{clk} is the clock period;
- $T_{\text{clk-to-q}}$ is the delay from a clock edge to flip-flop output change;
- T_{logic} is combinational logic delay;
- T_{route} is wire routing delay;
- T_{setup} is the time the destination input must be stable before the clock edge.

If this condition fails, there is a setup violation.

There is also a hold-time condition:

$$T_{\text{clk-to-q}} + T_{\text{logic,min}} + T_{\text{route,min}} \geq T_{\text{hold}}.$$

If data changes too quickly after the clock edge, the destination flip-flop may capture the wrong value. This is a hold violation.

Metastability

Metastability occurs when a flip-flop input is not stable around the sampling edge. The output may temporarily enter an undefined or oscillatory state. In digital design, metastability is dangerous because downstream logic assumes clean 0 or 1 values.

In this chip, timing risk is increased because some signals may need to travel from I/O near the top of the die down to compute logic and back up to output pins. Long routes increase delay and capacitance.

Slew and Capacitance

The lecture mentions slew violations. Slew refers to how quickly a signal transitions from low to high or high to low. Excessive capacitance can slow transitions.

A rough RC delay intuition is:

$$\tau = RC.$$

If wire capacitance C is large or driver resistance R is large, the transition becomes slow. Slow transitions can cause timing and signal-integrity problems.

Antenna Violations

During fabrication, long wires can accumulate charge during etching. If a wire is too long relative to the gate area it connects to, it can behave like an antenna and cause discharge damage. This is called an antenna violation.

The important practical lesson is:

Even if RTL simulation is correct, the physical chip may still fail because of timing, routing, capacitance, or manufacturability constraints.

Mathematical Reasoning

FLOP Count for 2×2 Matrix Multiplication

Each output element of a 2×2 matrix multiplication requires two multiply-add terms:

$$c_{ij} = w_{i0}x_{0j} + w_{i1}x_{1j}.$$

For one output element:

2 multiplications + 1 addition

is sometimes counted as 3 scalar operations. However, in hardware performance discussions, one multiply-accumulate is often counted as 2 FLOPs:

$$ab + c \Rightarrow 1 \text{ multiply} + 1 \text{ add} = 2 \text{ FLOPs.}$$

A 2×2 matrix multiplication has 4 output elements, each with 2 MACs:

$$4 \times 2 = 8 \text{ MACs.}$$

Thus the FLOP count is

$$8 \times 2 = 16 \text{ FLOPs.}$$

Baseline Throughput

The lecture states that one 2×2 matrix multiplication is completed every 8 cycles in the streaming steady state.

Therefore, average FLOPs per cycle are:

$$\frac{16 \text{ FLOPs}}{8 \text{ cycles}} = 2 \text{ FLOPs/cycle.}$$

Equivalently, since one MAC is 2 FLOPs,

$$2 \text{ FLOPs/cycle} = 1 \text{ MAC/cycle.}$$

With a 50 MHz clock,

$$50 \times 10^6 \text{ cycles/s} \times 2 \text{ FLOPs/cycle} = 100 \times 10^6 \text{ FLOPs/s.}$$

Thus,

$$\text{throughput} = 100 \text{ MFLOP/s.}$$

This is not large in absolute terms, but the point is architectural demonstration under microscopic area constraints.

Improved Weight-Stationary Throughput

With stationary weights and use of bidirectional pins for output, the chip can reduce the steady-state period from 8 cycles per tile toward 4 cycles per tile.

If one tile still represents 16 FLOPs, then

$$\frac{16 \text{ FLOPs}}{4 \text{ cycles}} = 4 \text{ FLOPs/cycle.}$$

At 50 MHz,

$$50 \times 10^6 \times 4 = 200 \times 10^6 \text{ FLOPs/s.}$$

So the improved scheme gives approximately

$$200 \text{ MFLOP/s.}$$

This is the source of the lecture's claim that better reuse and I/O use can double throughput.

Arithmetic Intensity

Arithmetic intensity is

$$\text{Arithmetic Intensity} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

For one baseline 2×2 multiply:

$$\text{input bytes} = 8,$$

because there are eight 8-bit inputs. The output is four 16-bit values:

$$\text{output bytes} = 4 \times 2 = 8.$$

Total traffic is approximately

$$8 + 8 = 16 \text{ bytes.}$$

The operation performs 16 FLOPs, so the arithmetic intensity is

$$\frac{16 \text{ FLOPs}}{16 \text{ bytes}} = 1 \text{ FLOP/byte.}$$

This is low. It means the design is highly sensitive to I/O bandwidth. Weight-stationary reuse increases arithmetic intensity because repeated use of a weight tile avoids repeated input bytes.

If a weight tile is reused R times, input traffic for R products is:

$$4 \text{ weight bytes} + 4R \text{ activation bytes.}$$

Output traffic is:

$$8R \text{ output bytes.}$$

The total is:

$$4 + 12R \text{ bytes.}$$

The total FLOPs are:

$$16R.$$

So arithmetic intensity becomes:

$$\text{AI}(R) = \frac{16R}{4 + 12R}.$$

As $R \rightarrow \infty$,

$$\lim_{R \rightarrow \infty} \frac{16R}{4 + 12R} = \frac{16}{12} = \frac{4}{3} \text{ FLOPs/byte.}$$

Even then, the design remains I/O-sensitive, but reuse improves the situation.

FP8 Number Representation

The floating-point version uses FP8 E4M3. This means:

- 1 sign bit;
- 4 exponent bits;
- 3 mantissa bits.

A normalized floating-point value is approximately

$$x = (-1)^s \cdot 2^{e-b} \cdot (1.m),$$

where:

- s is the sign bit;
- e is the encoded exponent;
- b is the exponent bias;
- m is the mantissa field.

For E4M3, the exponent bias is commonly

$$b = 7.$$

The lecture's practical point is not full IEEE compliance. The key point is that the chip uses a simplified FP8 multiply path and accumulates into an integer-like fixed-point representation before converting to BF16-style output.

Floating-Point Product

For two normalized floating-point values,

$$a = (-1)^{s_a} 2^{e_a - b} (1.m_a),$$

and

$$b = (-1)^{s_b} 2^{e_b - b} (1.m_b),$$

the product is

$$ab = (-1)^{s_a \oplus s_b} 2^{e_a + e_b - 2b} (1.m_a)(1.m_b).$$

To encode the product as a floating-point-like value with the same bias, the product exponent is conceptually

$$e_{\text{prod}} = e_a + e_b - b.$$

The TinyTPU avoids implementing a full floating-point multiply-add pipeline. Instead, it multiplies small mantissas and aligns products into a fixed-point accumulator.

Integer Accumulation

Instead of performing FP8 multiply followed by BF16 floating-point addition, the chip does a cheaper path:

FP8 $\times$ FP8 $\rightarrow$ fixed-point product $\rightarrow$ integer accumulator $\rightarrow$ BF16-like output.

The lecture mentions an 18-bit integer accumulator. Abstractly,

$$A^{(t+1)} = A^{(t)} + \text{align}(M_a M_b, e_a + e_b - b),$$

where M_a and M_b are small mantissa-derived integers and align shifts the product according to the exponent.

This is cheaper than full floating-point addition because floating-point addition requires:

- exponent comparison;
- mantissa alignment;
- signed addition or subtraction;
- normalization;
- rounding;
- special-case handling.

BF16 Conversion

BF16 has:

- 1 sign bit;
- 8 exponent bits;
- 7 mantissa bits.

To convert an integer accumulator to BF16-like form, the chip can use:

- sign extraction;
- leading-zero detection;
- normalization shift;
- mantissa bit selection;
- exponent construction.

If A is a positive fixed-point integer, a leading-zero detector finds

$$p = \lfloor \log_2 A \rfloor.$$

Then the normalized representation is roughly

$$A = 2^p \cdot q, \quad 1 \leq q < 2.$$

The BF16 exponent field becomes

$$e_{\text{BF16}} = p + b_{\text{BF16}},$$

where

$$b_{\text{BF16}} = 127.$$

The mantissa is formed from the leading fractional bits of q .

Code Walkthrough

Representative Verilog Processing Element

The lecture discusses Verilog RTL conceptually rather than presenting a full PE listing. A minimal representative integer PE is:

```
module pe_int8 (
    input wire      clk,
    input wire      reset,
    input wire      enable,
    input wire signed [7:0] w_in,
    input wire signed [7:0] x_in,
    output reg signed [7:0] w_out,
    output reg signed [7:0] x_out,
    output reg signed [15:0] acc
);

always @(posedge clk) begin
    if (reset) begin
        w_out <= 8'sd0;
        x_out <= 8'sd0;
        acc   <= 16'sd0;
    end else if (enable) begin
        acc   <= acc + w_in * x_in;
        w_out <= w_in;
        x_out <= x_in;
    end
end

endmodule
```

Important behavior:

- w_{in} and x_{in} are signed 8-bit values.
- The multiplication produces a wider product.

- The accumulator stores the local partial sum.
- w is forwarded horizontally through w_{out} .
- x is forwarded vertically through x_{out} .
- The PE updates only on a clock edge.

Hardware implications:

- The multiplier consumes significant area compared with simple control logic.
- The accumulator requires flip-flops.
- The forwarding registers create pipeline stages.
- The clock period must be long enough for multiply-add logic to settle.

Representative 2×2 Systolic Array

A simplified top-level array wires four PEs together:

```
module systolic_2x2 (
    input  wire clk,
    input  wire reset,
    input  wire enable,

    input  wire signed [7:0] w0_in,
    input  wire signed [7:0] w1_in,
    input  wire signed [7:0] x0_in,
    input  wire signed [7:0] x1_in,

    output wire signed [15:0] c00,
    output wire signed [15:0] c01,
    output wire signed [15:0] c10,
    output wire signed [15:0] c11
);

wire signed [7:0] w00_to_01;
wire signed [7:0] w10_to_11;
wire signed [7:0] x00_to_10;
wire signed [7:0] x01_to_11;

pe_int8 pe00 (
    .clk(clk), .reset(reset), .enable(enable),
    .w_in(w0_in), .x_in(x0_in),
    .w_out(w00_to_01), .x_out(x00_to_10),
    .acc(c00)
);
```

```

pe_int8 pe01 (
    .clk(clk), .reset(reset), .enable(enable),
    .w_in(w00_to_01), .x_in(x1_in),
    .w_out(), .x_out(x01_to_11),
    .acc(c01)
);

pe_int8 pe10 (
    .clk(clk), .reset(reset), .enable(enable),
    .w_in(w1_in), .x_in(x00_to_10),
    .w_out(w10_to_11), .x_out(),
    .acc(c10)
);

pe_int8 pe11 (
    .clk(clk), .reset(reset), .enable(enable),
    .w_in(w10_to_11), .x_in(x01_to_11),
    .w_out(), .x_out(),
    .acc(c11)
);

endmodule

```

This code illustrates the spatial mapping:

$$PE_{00}, PE_{01}, PE_{10}, PE_{11}$$

produce

$$c_{00}, c_{01}, c_{10}, c_{11}.$$

In the real chip, the input timing is carefully controlled so that each PE receives the right w and x values in the right cycle.

Representative Streaming Driver

The software or testbench must feed the chip according to the expected row-major loading order. A simplified Python-style driver is:

```

def run_tile(tpu, W, X):
    values = [
        W[0][0], W[0][1],
        W[1][0], W[1][1],
        X[0][0], X[0][1],
        X[1][0], X[1][1],
    ]

```

```

]

for value in values:
    tpu.write_input(value)
    tpu.tick()

outputs = []
for _ in range(8):
    byte = tpu.read_output()
    outputs.append(byte)
    tpu.tick()

return decode_four_int16(outputs)

```

The critical detail is that the software interface must obey the hardware protocol. The chip is not discovering matrix shapes dynamically. Its control logic expects a fixed order.

Representative Cocotb Test

The lecture discusses Cocotb for functional verification. A representative test-bench is:

```

import cocotb
from cocotb.triggers import RisingEdge

async def reset_dut(dut):
    dut.reset.value = 1
    for _ in range(2):
        await RisingEdge(dut.clk)
    dut.reset.value = 0
    await RisingEdge(dut.clk)

@cocotb.test()
async def test_2x2_matmul(dut):
    await reset_dut(dut)

    W = [[1, 2],
          [3, 4]]
    X = [[5, 6],
          [7, 8]]

    expected = [[19, 22],
                 [43, 50]]

    await drive_tile(dut, W, X)
    observed = await collect_tile(dut)

```

```
assert observed == expected
```

Cocotb uses Python coroutines. The test waits for clock edges while the Verilog simulator advances the hardware model. This allows the test to manipulate pins cycle by cycle.

Quantization-Aware Training in PyTorch

The lecture describes using PyTorch 2 Export quantization tools. A representative version is:

```
import torch
import torch.nn as nn

class SmallMLP(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc1 = nn.Linear(784, 128)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = self.fc1(x)
        x = self.relu(x)
        x = self.fc2(x)
        return x

model = SmallMLP()
example_inputs = (torch.randn(1, 784),)

exported = torch.export.export(model, example_inputs)
```

The exported graph can then be prepared for quantization-aware training. Conceptually, QAT inserts quantize and dequantize nodes:

$$x \rightarrow \text{quantize}(x) \rightarrow \text{dequantize}(q) \rightarrow \text{linear layer}.$$

This simulates information loss during training.

Affine Quantization

For integer quantization, a common formula is:

$$q = \text{clamp} \left(\text{round} \left(\frac{x}{s} \right) + z, q_{\min}, q_{\max} \right),$$

where:

- x is the real-valued tensor element;
- s is the scale;
- z is the zero point;
- $q_{\min}$ and $q_{\max}$ define the integer range.

Dequantization is:

$$\hat{x} = s(q - z).$$

Quantization-aware training trains the model while these effects are present, so the model learns parameters that are robust to low-precision inference.

PyTorch Backend Integration

The speaker initially called the TinyTPU matrix-multiplication kernel manually for each layer. This was cumbersome. A better solution is to integrate the TinyTPU operation into a PyTorch graph backend.

PyTorch linear layers compute:

$$Y = XA^T + b.$$

This is important because TinyTPU supports fused transpose of the second operand. Thus, the PyTorch linear convention maps naturally to the chip's fused transpose mode.

A representative custom operation wrapper is:

```
import torch

@torch.library.custom_op("tinytpu::matmul", mutates_args=())
def tinytpu_matmul(x: torch.Tensor,
                   w: torch.Tensor) -> torch.Tensor:
    return call_tinytpu_kernel(x, w)
```

A custom backend can traverse the graph and replace recognized linear operations with this custom operator.

Conceptually:

$$\text{linear}(x, w, b) \rightarrow \text{tinytpu_matmul}(x, w) + b.$$

If quantization nodes are present, the backend may also fuse or remove unnecessary quantize-dequantize operations around the custom kernel.

Performance Intuition

The Chip Is Mostly I/O-Limited

The compute core is small, but even the small 2×2 systolic array wants more input bandwidth than the platform exposes. The key mismatch is:

$$32 \text{ desired input bits/cycle} > 8 \text{ available input bits/cycle.}$$

Therefore, throughput is governed by how well computation can be overlapped with input and output movement.

This is similar to GPU kernel optimization. A CUDA kernel may have many arithmetic units available, but if global memory bandwidth or memory access pattern is poor, compute units sit idle.

Data Reuse Is Everything

The lecture's weight-stationary improvement is the same idea as tiling in CUDA shared memory. In CUDA GEMM, a block loads a tile of A and B into shared memory so multiple threads can reuse them.

The same principle appears here:

$$\text{load once} \Rightarrow \text{reuse many times} \Rightarrow \text{higher arithmetic intensity.}$$

On GPUs, reuse reduces global-memory traffic. On TinyTPU, reuse reduces pin I/O traffic.

Fusion Saves Bandwidth

Fused transpose and fused ReLU are useful because they avoid separate passes. In bandwidth-limited systems, even cheap operations can be expensive if they require extra memory movement.

For ReLU, separate execution would require:

$$\text{read } C + \text{write } Y.$$

Fused execution performs the comparator before output, avoiding a separate storage and reload step.

Odd Matrix Dimensions Hurt Utilization

The chip operates on 2×2 tiles. If a larger matrix has odd dimensions, padding is required. For example, a 3×3 matrix may be padded to 4×4 .

The useful fraction is:

$$\frac{3 \times 3}{4 \times 4} = \frac{9}{16} = 56.25\%.$$

The remaining work is wasted on zeros. This is analogous to GPU tile quantization effects, where block sizes and tensor dimensions interact.

Why FP8 Can Be Cheaper Than INT8 in This Design

The speaker observed that implementing the FP8-style arithmetic path was cheaper than expected and, in some ways, cheaper than full 8-bit integer arithmetic. The reason is that E4M3 mantissas are very small.

An INT8 multiplier multiplies two 8-bit integers. A simplified FP8 mantissa multiply may multiply much smaller mantissa fields, such as 3-bit or 4-bit effective values, then use exponent shifts.

The area trade-off is:

small mantissa multiply + shift/exponent logic versus full INT8 multiply.

In a tiny design, the exact synthesis result can favor the simplified FP8 path.

Why Full Floating-Point Addition Is Expensive

Full floating-point addition is much more complex than integer addition. To compute

$$a + b,$$

hardware must:

1. compare exponents;
2. shift the smaller mantissa;
3. add or subtract mantissas depending on signs;
4. normalize the result;
5. round the result;
6. handle overflow, underflow, zeros, infinities, and NaNs if supported.

The TinyTPU avoids this by accumulating products in a fixed-point-like integer accumulator.

Comparison with GPU Tensor Cores

Modern NVIDIA tensor cores also use low-precision inputs with higher-precision accumulation. A common pattern is:

FP16/BF16/FP8 inputs $\rightarrow$ higher precision accumulation $\rightarrow$ rounded output.

The TinyTPU reproduces the same broad design principle at a toy scale:

FP8 inputs $\rightarrow$ INT18-style accumulation $\rightarrow$ BF16-style output.

The lesson is that low-precision ML hardware is not simply about storing fewer bits. It is about choosing where precision is needed:

- low precision for bandwidth and multiply area;
- higher precision for accumulation stability;
- controlled rounding or conversion at boundaries.

Common Misconceptions

Misconception: A TPU Is Only a Big Matrix Multiplier

A TPU is not merely a multiplier. Even this tiny TPU needs:

- local storage;
- a dataflow schedule;
- I/O protocol;
- control signals;
- verification infrastructure;
- software kernels;
- framework integration.

The matrix multiplier is the heart, but the surrounding system determines whether it is usable.

Misconception: RTL Correctness Means the Chip Will Work

RTL simulation only checks logical behavior under an abstract timing model. Real silicon also needs:

- setup timing closure;
- hold timing closure;
- slew checks;
- antenna checks;
- routing feasibility;
- manufacturability compliance.

A chip can be functionally correct in simulation and still fail physically.

Misconception: Larger Systolic Arrays Are Just Repetition

A larger systolic array increases:

- PE count;
- operand bandwidth requirements;
- accumulator storage;
- clock-tree load;
- routing congestion;
- verification complexity;
- power delivery complexity.

Scaling from 2×2 to 128×128 is not just copy-paste.

Misconception: Fused Operations Are Only Software Optimizations

Fusion is also a hardware optimization. A fused transpose or fused ReLU can avoid moving data through external I/O or memory. On a bandwidth-limited chip, saving movement may matter more than saving arithmetic.

Misconception: INT8 Is Always Cheaper Than FP8

INT8 has simple semantics, but an 8×8 integer multiplier can still be expensive. FP8 E4M3 has small mantissas, so a simplified multiply path can be surprisingly compact if special cases are limited.

Misconception: Quantization Is Only a Post-Training Compression Trick

Quantization-aware training changes the training process by simulating quantization noise during training. The model learns to tolerate low-precision inference.

Practical Takeaways

- A real accelerator is a full stack: silicon, RTL, dataflow, verification, kernels, and framework integration.
- Systolic arrays are valuable because they turn matrix multiplication into structured local communication.
- I/O bandwidth can dominate performance even when compute logic is available.
- Streaming schedules overlap input, compute, and output to improve utilization.
- Weight-stationary dataflow increases reuse and reduces repeated loading.
- Fused operations are valuable when they eliminate data movement.
- Low-precision arithmetic must balance multiplier cost, accumulator precision, and output format.
- Fixed-point or integer accumulation can avoid the high area cost of full floating-point addition.
- Functional testing with Cocotb is necessary but not sufficient; physical timing and manufacturability checks are also required.
- PyTorch integration requires graph-level thinking, not only kernel-level thinking.

Project or Experiment Ideas

Experiment 1: Simulate a 2×2 Systolic Array

Implement a cycle-accurate Python simulator for a 2×2 systolic array. Track the values inside each PE at every cycle.

Measure:

- number of cycles per tile;
- PE utilization;
- effect of different input schedules;
- correctness for random integer matrices.

Experiment 2: Compare Dataflows

Implement three dataflows for blocked matrix multiplication:

- output stationary;
- weight stationary;
- activation stationary.

For each, count bytes moved and compute arithmetic intensity:

$$\text{AI} = \frac{\text{FLOPs}}{\text{Bytes Moved}}.$$

Experiment 3: Build a Cocotb Testbench

Write Verilog for a tiny PE and a 2×2 array. Use Cocotb to test:

- reset behavior;
- one matrix multiplication;
- random matrix multiplication;
- fused ReLU;
- fused transpose.

Experiment 4: Implement FP8 Approximate Arithmetic

Implement E4M3 encoding and decoding in Python. Then compare:

FP8 multiply + FP32 accumulation

against

FP8 multiply + fixed-point accumulation + BF16 conversion.

Measure absolute error:

$$|y_{\text{approx}} - y_{\text{reference}}|$$

and relative error:

$$\frac{|y_{\text{approx}} - y_{\text{reference}}|}{|y_{\text{reference}}| + \epsilon}.$$

Experiment 5: Quantization-Aware Training

Train a small MLP on MNIST or Fashion-MNIST using PyTorch quantization-aware training. Compare:

- FP32 inference accuracy;
- post-training INT8 quantization accuracy;
- quantization-aware INT8 accuracy.

Experiment 6: PyTorch Graph Replacement

Use `torch.export` or `torch.fx` to trace a small model. Replace linear layers with a custom Python function that calls a simulated TinyTPU backend.

Verify that

$$Y = XA^T + b$$

is preserved numerically.

Final Mental Model

TinyTPU is a complete miniature accelerator stack. At the bottom, it is a few processing elements connected as a 2×2 systolic array. Each PE performs multiply-accumulate and forwards operands to neighbors. Around that array, the chip needs local buffers, control signals, a streaming schedule, and careful use of narrow I/O pins.

The most important bottleneck is not the multiply itself. The bottleneck is moving enough data through a tiny pin interface to keep the array useful. The main architectural tricks are therefore streaming, overlap, reuse, stationary weights, fused transpose, and fused ReLU.

At the arithmetic level, the chip shows why modern ML hardware often uses low-precision inputs with higher-precision accumulation. FP8 saves area and bandwidth, while INT18 or BF16-style accumulation protects numerical stability.

At the software level, the chip becomes useful only when matrix multiplication kernels are tiled, quantized models are trained, and PyTorch graphs are rewritten so ordinary linear layers can target the custom hardware.

The lecture's core lesson is that an accelerator is not just a fast datapath. It is a negotiated design across area, I/O bandwidth, numerical format, dataflow, verification, and compiler or framework integration.